Bashir Agro Admin Panel
Phase 11: API Preparation Module

Scope

Prepare the Laravel backend for future Flutter Farmer App integration.

Included:
- API architecture preparation
- Backend restructuring
- Authentication preparation
- Permission planning
- API response planning
- Resource preparation
- Service layer improvement
- Documentation
- Testing preparation

Not included:
- Actual REST API
- Flutter app
- Farmer login API
- Task submission API
- Photo upload API
- GPS API
- PostGIS

Objective

Convert the Laravel Admin Panel backend into a structure that can support both:

1. Existing Admin Panel
2. Future Flutter Farmer Application

Both systems will share the same Laravel backend and PostgreSQL database.

Architecture

Current:

Controller
|
Model
|
Database

Target:

Controller
|
Service Layer

|
Query/Repository Layer
|
Models
|
Database

Business logic should not stay inside controllers because APIs will reuse it later.

API Structure Preparation

Create structure:

app/Http/Controllers/Api/V1/

Future controllers:

AuthController
FarmerController
TaskController
AssignmentController
SubmissionController

Do not implement endpoints yet.

API Versioning

Prepare:

/api/v1/

Future examples:

GET /api/v1/tasks
POST /api/v1/login
GET /api/v1/profile

Versioning allows future API upgrades without breaking old apps.

Authentication Preparation

Prepare Laravel Sanctum.

Future flow:

Flutter App
|
Bearer Token
|
Laravel Sanctum
|
User

Do not create login API in this phase.

User and Farmer Preparation

Prepare relationship:

User
|
|
Farmer Profile

Future roles:

owner
admin
farmer

Farmer mobile access will use this structure.

Permission Planning

Prepare permission structure.

Examples:

Admin:
farmers.manage
locations.manage
tasks.create
assignment.create
reports.view

Farmer:
task.view
task.submit
profile.view

Recommended:
spatie/laravel-permission

API Resources

Prepare:

app/Http/Resources/

Future:

FarmerResource
TaskResource

AssignmentResource
LocationResource

Purpose:
Control exactly what mobile applications receive.

Database Review

Review existing modules:

Farmers:
- profile
- status
- user relationship

Locations:
- coordinates
- radius

Tasks:
- instructions
- photo rules
- geofence requirement

Assignments:
- farmer
- task
- location
- schedule
- status

Service Layer Improvement

Move reusable logic into services.

Example:

TaskAssignmentService

Functions:

createAssignment()
updateStatus()
cancelAssignment()

Used by:
Admin Panel
Future API

Storage Preparation

Prepare:

storage/app/public/

Folders:

farmers/
tasks/
submissions/

Future use:
profile photos and task evidence images.

Documentation

Create:

docs/

API_ARCHITECTURE.md
DATABASE_SCHEMA.md
MOBILE_FLOW.md

Document:
- authentication
- relationships
- permissions
- mobile workflow

Testing Preparation

Prepare:

tests/Feature/Api/

Future tests:

AuthApiTest
TaskApiTest
AssignmentApiTest

Security Preparation

Prepare:

- Sanctum authentication
- Authorization policies
- Rate limiting
- Validation reuse
- Secure API responses

Routes Preparation

Prepare:

routes/api.php

Structure:

```
Route::prefix('v1')
->group(function(){

});
```

Do not add API endpoints yet.

Completion Checklist

Verify:

✓ Admin Panel still works
✓ Database remains stable
✓ Services created
✓ API structure created
✓ Authentication strategy prepared
✓ Resources prepared
✓ Documentation created
✓ Tests structure created

Antigravity Instruction

Proceed to PHASE 11 ONLY: API Preparation Module.

Use the existing Bashir Agro Laravel project after Phase 10.

This phase is preparation only.

Do NOT build actual APIs.

Do NOT implement:
- Flutter app
- Login API
- Task API
- Submission API
- Photo upload API
- GPS API
- PostGIS

Implement:

- API folder structure
- Versioning preparation
- Sanctum preparation
- Role and permission preparation
- API Resources preparation
- Service layer improvement
- Storage preparation

- Documentation
- Testing structure

Preserve:
- Existing Admin Panel
- Existing UI
- Existing functionality

Run:

php artisan test

vendor/bin/pint

Report:
- Files created
- Files modified
- Database changes
- Architecture changes
- Test results

STOP after Phase 11.
Do not start Phase 12 automatically.